

ESP32 Wi-Fi Timing Gates: A Hardware-Synchronized Architecture

ESP32 Wi-Fi Timing Gates: A Hardware-Synchronized Architecture

Abstract - Distributed timing architectures deployed across wireless networks typically suffer from variable network transport latency and operating system jitter. This paper presents **TARTARUS**, a hardware-synchronized, microsecond-accurate robotic timing gate ecosystem built on the ESP32 platform. Rather than relying on standard network time synchronization protocols, the system anchors remote nodes to a centralized hardware clock timeline by capturing raw Timing Synchronization Function (TSF) timestamps embedded directly within local Wi-Fi beacon frames. To mitigate wireless stack resets and airtime contention, the edge firmware (**CERBERUS**) implements a multi-threaded FreeRTOS queue architecture decoupled from a secondary, autonomous self-healing processor clock audit engine. Empirical field trials across a dedicated local access point have demonstrated a system-wide pairing reliability of 100% over extended operational windows, maintaining a mean tracking accuracy gap of $2.36\ \mu\text{s}$ and a worst-case maximum variance of $55\ \mu\text{s}$.

TARTARUS is a highly resilient, distributed hardware-synchronized timing ecosystem engineered for high-performance robotic track automation and precision race scoring. At its core, the system shifts the burden of microsecond synchronization away from complex software algorithms and hands it directly to the physical layer of your wireless infrastructure. By leveraging the native 1 MHz Timing Synchronization Function (TSF) counters broadcast inside standard Wi-Fi beacon frames, the system establishes an immutable, synchronized field timeline. Whether tracking rapid-fire beam breaks on the track surface or bridging asynchronous network outages across the wireless spectrum, this architecture ensures that timing metrics are captured instantly, buffered securely, and recorded accurately. This document outlines the structural framework, naming conventions, firmware logic, and backend configurations that allow the system to maintain a sub-100 microsecond timing accuracy without physical connection cables.

System Hierarchy & Component Roles

To navigate this documentation, the architecture is broken down into six discrete layers named after classical mythology (because I could), establishing a clear chain of command from physical track sensors to the master scoreboard:

- **TARTARUS** (*Timing Architecture for Remote Track Automation, Recording, and Unified Scoring*): **The Overall Ecosystem**. Represents the complete network topology, the combined codebases,

and the mathematical clock calibration models working as one unified system.

- **HADES** (*Host Automation, Detection, Event Scoring*): **The Central PC Software**. The lord of the central network; it hosts the database, runs the retrospective sliding-window pairing algorithms, and maintains the official scoreboard.
- **CHARON** (*Central Hub and Radio Interface for Over-the-air Node Synchronization*): **The Wi-Fi Access Point**. The absolute ruler of time on the track. Through hardware TSF beacon frames, it dictates the master timeline and physically ferries data packets across the air.
- **CERBERUS** (*Coordinated ESP Radio Bridge for Event Recording and Untethered Scoring*): **The Gate Controller**. The multi-threaded FreeRTOS application monitoring the gates. It handles hardware interrupts, executes continuous clock sanity audits, and caches events in RAM during network drops. It collates raw gate notifications into clean split and lap times. It can act as a stand-alone scoring device if needed.
- **HESPERUS** (*Hardware for ESP32 Sensor Precision Entry and Return Signals*): **The Physical Sensor Gates**. The independent gate hardware and edge software. They control and monitor the individual maze/track sensors, tag events with microsecond-accurate timestamps and sends notifications to the gate controller.
- **HERMES** (*Hyper-accurate ESP32 Remote Measurement Exchange System*): **The Communication Protocol**. The application-layer HTTP packet layout that swiftly carries microsecond timestamps from the individual HESPERUS gates to CERBERUS for processing into event times..

Introduction

Wi-Fi Access Points (APs) generate beacon frames to advertise their presence approximately 10 times per second. While this rate is not strictly guaranteed - especially in congested wireless environments - it would be highly unusual not to receive a beacon frame at least once per second.

Among the critical metadata contained within a standard beacon frame is the **Timing Synchronization Function (TSF)** timestamp.

The TSF keeps the internal timers of all client stations (STAs) within the same Basic Service Set Identifier (BSSID) - meaning those connected to the same access point - in close synchronization. The AP and each connected station maintain an internal 64-bit counter that increments at 1 MHz. Left unmanaged, these individual clocks would inevitably drift apart due to differences in local crystal oscillators.

However, whenever a station intercepts a beacon frame, it automatically updates its local TSF counter with the authoritative value provided by the AP. This hardware-level mechanism ensures that all stations maintain an internal clock likely to match within a few tens of microseconds, and frequently within less than ten microseconds.

With 64 bits of headroom, this microsecond counter will not wrap around for more than 500,000 years. While the absolute value of the counter holds no useful meaning on its own - typically resetting to zero

whenever the AP reboots - its relative value across multiple units is highly valuable for distributed event tracking.

Sources of Error

Maintaining the local TSF counter is handled entirely at the silicon and network-stack layer; client software requires no manual overhead to ensure correct operation. As noted, although beacon frames may arrive irregularly due to airtime contention, an individual station's clock will not drift by more than a few microseconds over a one-second window.

However, capturing this value in software introduces real-world variables:

- **Operating System Jitter:** In standard ESP32 deployments, the Wi-Fi drivers run on FreeRTOS, with the core wireless network components typically bound to Core 0. Because FreeRTOS may be executing an alternate application task at the exact millisecond a physical event occurs, task-switching overhead can introduce a latency of several microseconds.
- **Hardware Interrupt Latency:** Even when bypassing FreeRTOS task scheduling by deploying a direct hardware interrupt service routine (ISR) to detect the gate trigger, a small execution latency remains. Furthermore, fetching the internal counter register itself takes a couple of microseconds. This latency can be drastically minimized by utilizing the `IRAM_ATTR` macro, forcing the compiler to place the ISR code directly into fast internal RAM rather than executing it out of slow flash memory.
- **Single-Core Constraints:** Single-core variants, such as the ESP32-C3, must process all application code and Wi-Fi drivers on the same CPU core. This inherently increases worst-case jitter compared to dual-core architectures like the standard ESP32 or ESP32-S3. To safeguard timing accuracy on single-core variants, it is highly recommended to configure **High-Priority Interrupts** (Level 2 or above) to allow the timing gate to preempt active network stack processes.

Because the counter remains easily readable in code, it serves as an ideal, immutable timestamp for any local client event. Since all stations locked to the same AP share a synchronized hardware counter, the system can reliably determine the exact time delta between widely separated clients to the identical degree of precision.

While individual client latencies and hardware jitter add up mathematically when calculating the true period between remote events, employing standard embedded best practices makes it relatively easy to guarantee an overall system error of less than $100\ \mu\text{s}$, and frequently better than $50\ \mu\text{s}$.

Once the TSF counter value is latched during an event, the remaining application code can take a relaxed approach to processing and transmitting the data - provided a rapid succession of events does not overwhelm the buffer. Even in high-frequency scenarios, a standard FreeRTOS queue can safely buffer timestamps and event IDs to be systematically offloaded by a secondary networking task. The only strict rule is that the long-term average rate of incoming events does not exceed the network's average transmission throughput.

Power Management

Running an ESP32 with its radio continuously enabled draws more current than is preferable for a long-term, battery-powered field deployment. For example, an ESP32-S3 with an active radio transmitting short HTTP requests once per second will draw an average of 50 to 70 mA.

If this power load is too steep for the target battery capacity, the remote station can be configured to use **Wi-Fi Modem Sleep**. In this mode, the power-hungry RF circuitry is placed into a low-power state between beacon intervals while keeping the internal hardware TSF counter fully operational. The chip automatically wakes up its receiver just in time to listen for the scheduled beacon frame - a task it coordinates using the TSF clock itself, which is exactly what the feature was natively designed to do.

Utilizing Wi-Fi Modem Sleep can drop average current draw down to as little as a quarter of standard active listening mode. Meanwhile, the localized event detection logic continues to operate unimpeded, instantly waking the radio to transmit logged events before returning the chip safely back to sleep.

Reporting Events

The remote timing gates act as decentralized, battery-operated nodes. Because they rely on a central Wi-Fi access point for microsecond-accurate time synchronization, it is highly practical to use that same established wireless link to report timing data back to a central master station.

These data reports are encapsulated within Transmission Control Protocol (TCP) packets. A defining feature of TCP, compared to the simpler User Datagram Protocol (UDP), is its connection-oriented nature: packets cannot silently vanish without the sender's explicit knowledge. Once sent, the TCP protocol automatically manages handshakes, sequencing, and retransmissions, guaranteeing successful delivery or providing an immediate notification of hardware connection failure.

That absolute transport reliability comes at a chronological cost: there is no obvious way to predict exactly how long a TCP packet takes to arrive and be processed by the receiver, especially if wireless congestion forces packet retransmissions.

However, because the system embeds the locally latched 64-bit TSF event timestamp directly into the packet payload, variable network transport latency becomes completely irrelevant. Whether the message takes 5 milliseconds to clear the air or is delayed by a major network glitch for 500 milliseconds, the enclosed timestamp tells the receiver precisely when the physical event occurred.

Application Layer: HTTP

Knowing that the Wi-Fi stack guarantees data delivery via TCP satisfies only part of the development lifecycle; designing custom, robust communication protocols from scratch is a complex and error-prone engineering challenge. It is far more efficient to layer a highly common, well-established application protocol on top of TCP. For this system, the most logical choice is Hypertext Transfer Protocol (HTTP).

Using a foundational HTTP `GET` request, a remote gate packages its data into a simple, human-readable text URL string containing parameters such as the Gate ID, the microsecond-accurate timestamp, the target BSSID, the calculation clock mode, and the trigger type string. This string is transmitted to a local web server at a known address. The web server parses the incoming request, handles the underlying database or timing logic, and returns a standard response.

This model establishes a double layer of operational guarantees:

1. **The Transport Layer (TCP):** Confirms the physical data packets arrived at the destination network stack.
2. **The Application Layer (HTTP):** Employs standard status codes (e.g., `200 OK`) to confirm that the server software successfully received, interpreted, and validated the data message.

As an added benefit, the server's HTTP reply can pass custom text payloads back to the remote gate, providing a built-in channel for remote commands, status updates, or system diagnostics.

To ensure the remote gate's radio is not powered up any longer than necessary, messages should be kept highly concise. While creating and tearing down a standard TCP/HTTP connection involves a handful of auxiliary overhead packets, the entire transaction should still conclude within a few milliseconds under normal conditions.

Web Server

With the remote gates successfully capturing and communicating high-accuracy timestamps, the central collecting device functions purely as a web server. In principle, this server could reside anywhere in the world via the internet. Practically, however, it should be deployed locally on the exact same Wi-Fi subnet. A local topology completely eliminates external internet overhead, removes reliance on cellular or WAN connectivity in the field, and ensures a highly secure, closed ecosystem.

The local collecting web server can be tailored precisely to the scope of the event. At its most basic, it can comprise a simple Python script running on a Raspberry Pi or a low-power laptop. Alternatively, it can run efficiently on another microcontroller, such as an ESP32-based **Cheap Yellow Display (CYD)** programmed in C++.

Depending on available processing resources, this web server can host the entire race-management system independently. For more lightweight hardware implementations, it can serve as a local bridge controller, parsing the incoming Wi-Fi messages and streaming them over a physical serial port to a host computer running comprehensive desktop scoring software.

A useful side-effect of deploying an HTTP web server to collect and process data is that other devices on the same local network can seamlessly query the server for parallel applications. Depending on the server's backend configuration, it can concurrently stream data to local secondary displays to show live runtimes, or populate an interactive, real-time leaderboard for spectators and officials.

Provisioning

A dedicated local Wi-Fi network is easy to establish using almost any inexpensive, commercial domestic router configured as a standalone Access Point. The router handles broadcasting the BSSID, managing a clean wireless channel, dynamically allocating local IP addresses to clients, and ensuring seamless device intercommunication.

A dedicated commercial router will offer significantly better antenna arrays and RF performance than a software-defined access point hosted on an embedded computer like a Raspberry Pi. While a single Raspberry Pi *could* technically act as both the network AP and the collecting web server, separating these roles into distinct hardware enclosures enhances overall system resilience and flexibility - even if it makes the physical field setup slightly more burdensome.

Network addressing requires careful design. The remote gates must know the exact IP address of the web server to direct their HTTP requests. Conversely, the web server does not need prior knowledge of the gates' IP addresses because every incoming HTTP message payload includes an identifying token. The management software can dynamically map that unique identifier to a physical track location, such as "Start Gate" or "Finish Gate".

Consequently, the web server must be assigned a fixed, unmoving IP address, which is easily configured on any standard access point using a static IP mapping or a DHCP reservation.

An elegant alternative is utilizing a dynamic local protocol such as **multicast DNS (mDNS)**. Under this architecture, the web server advertises its friendly name on the local subnet (e.g., `http://timing.local`). The remote gates locate the server dynamically by its hostname rather than a hardcoded numerical address, making the entire ecosystem truly plug-and-play. Support for mDNS is natively built directly into the core ESP32 Wi-Fi stack.

Initial Baseline Testing

To evaluate empirical system reliability, an isolation test was executed using two identical ESP32-S3 boards connected to a shared 1PPS (Pulse Per Second) hardware signal generator. The boards acted as simulated timing gates, generating an HTTP `GET` request containing a copy of the locally latched TSF counter on every incoming hardware edge.

A host PC running on the same local Wi-Fi network executed a Python script acting as the collection web server. The script evaluated pairing metrics by matching concurrent entries from both gates. A valid pair was defined as two incoming requests (one from each gate) whose internal TSF timestamps differed by less than 200 ms. This 200 ms threshold provides a safety buffer orders of magnitude wider than any expected network transmission variance generated by a simultaneous event.

Under this framework, pairing failures indicate lost data payloads or server-side processing bottlenecks. Because the requests travel over TCP, genuine packet drops are managed automatically by the lower-level network interface card (NIC) stacks, making complete message dropouts exceedingly rare.

Empirical Results and Discovery

Following 14 hours of continuous operation across a standard domestic Wi-Fi network comprising 52,200 successful pairs, the system logged **121 failed matches**. While this corresponds to a seemingly minor 0.23% error rate, an ideal hardware-synchronized architecture requires zero systematic failures.

Forensic analysis of the server logs revealed three distinct root causes behind these missed pairings:

- 1. Server-Side Thread Bottlenecks:** Standard Python environments are bound by the Global Interpreter Lock (GIL). A combination of console logging bottlenecks and thread scheduling latency can cause the script to momentarily stall, failing to process sequential incoming data within the matching loop window.
- 2. Network Layer TCP Retransmission Lag:** In a contested RF environment, momentary Wi-Fi link degradation forces the lower network stack to enter automatic retry loops. While the data is never lost, it can be delayed anywhere from 200 to 1000 ms. By the time the delayed request reaches the server, its partner has already been evaluated and flagged as unmatched.
- 3. Wi-Fi Subsystem Resets (The Primary Error Source):** The single most disruptive anomaly observed was a localized clock reset within the ESP32 Wi-Fi stack. If a client station misses consecutive AP beacon frames or undergoes an internal radio re-association, the network stack temporarily uninitialized the virtual interface. Upon recovery, the local TSF register is cleared to **zero** while the hardware waits for a fresh beacon frame to resynchronize its timeline. Events recorded during this recovery window contain an invalid TSF value of zero, breaking the server's pairing algorithms and creating cascading tracking errors.

High-Frequency Gate Handling & Time Recovery

A single gate board may support multiple asymmetrical trigger inputs. For example, a start cell controller must process both a proximity/presence detector and an explicit start sensor. Consider a high-performance robot returning to a start cell: accelerating or decelerating at 20 m/s^2 allows a vehicle traveling at 2.5 m/s to come to a complete stop within 168 mm. Under these real-world mechanics, the robot will trip the presence sensor a mere 35 ms after cross-cutting the primary start beam.

To handle rapid-fire bursts on the order of a few tens of milliseconds, the gate must decouple physical event recording from data transmission. Pin interrupts must be aggressively debounced within a safe hardware-compatible lockout window (20 ms or 20,000 μs) to avoid missing back-to-back entries while throwing out mechanical contact noise.

Leveraging FreeRTOS, the hardware interrupt service routine (ISR) instantly captures telemetry metrics, packages them into a `GateEvent` data structure, and pushes them onto an internal **FreeRTOS Queue** (`networkQueue`) sizing up to 10 events. A detached network worker task running at an elevated priority (Priority 2) systematically processes this queue at its own pace. This design allows

the edge node to capture bursts of events at microsecond intervals, streaming them out over HTTP as network bandwidth permits without stalling the physical sensors.

Autonomous Self-Healing Clock Recovery

To completely eliminate the temporal errors caused by Wi-Fi stack resets and zeroed counters, the edge firmware employs a continuous timeline audit utilizing a secondary, independent hardware clock: the internal ESP32 64-bit monotonic processor timer (`esp_timer_get_time()`).

Unlike the volatile TSF clock, this processor clock is driven directly by the internal CPU crystal oscillator. It ticks continuously at 1 MHz from the exact microsecond of boot, remaining completely unaffected by radio dropouts, channel switches, or network re-associations. Because both clocks increment at an identical 1 MHz frequency, their relationship is linear and behaves according to a predictable offset.

The self-healing algorithm operates as follows:

1. **Simultaneous Capture:** The hardware ISR captures both the `tsf_observed` value and the `processor_clock` value at the exact microsecond the physical pin drops.
2. **Continuous Sanity Auditing:** On every incoming event, the background worker task calculates the elapsed time since the previous event using the rock-solid processor clock. Multiplying this elapsed time by a dynamically adjusted tracking scale factor (`clock_alpha`) and adding it to the previously recorded TSF baseline establishes an **expected TSF value**.
3. **Drift Evaluation:** The firmware compares the expected TSF value against the raw observed TSF value. If the observed clock matches the expected timeline within a strict pre-determined tolerance margin of $\leq 500 \mu s$ (`DRIFT_MARGIN_US`), the Wi-Fi stack is verified healthy.
4. **Live Runtime Calibration:** When the timeline is healthy, if more than 4 seconds have passed since the last step, an updated snapshot is taken. The firmware calculates the instantaneous drift ratio between the actual TSF delta and the processor delta. This is fed into an Exponential Moving Average (EMA) filter using a 10% weight coefficient (`EMA_ALPHA = 0.10`) to update `clock_alpha`, smoothing out localized network beacon jitter:

$$\text{clock_alpha} = (0.10 \times \text{instant_alpha}) + (0.90 \times \text{clock_alpha})$$

5. **Autonomous Synthesis:** If the observed TSF clock reads zero or drifts outside the permitted $500 \mu s$ threshold - indicating an active Wi-Fi subsystem reset - the firmware rejects the stack value. It flags the packet mode as `"SYN"` and seamlessly reconstructs the true timestamp using the current monotonic processor clock extrapolated from the last known stable baseline coordinates:

$$\text{TSF}_{\text{synthesized}} = \text{TSF}_{\text{last_good_observed}} + (\text{ElapsedProcessorClock} \times \text{clock_alpha})$$

6. **Circuit Breaker Recovery:** If the router timeline undergoes a genuine hardware step or permanent plateau shift, the audit engine will detect consecutive failures. If the discrepancy persists for 5

consecutive cycles, the firmware triggers an override recovery mechanism, accepting the new router baseline shift as valid and resetting calibration parameters to clear the lockdown.

Transport Resiliency and Application Timeouts

While the edge-level clock recovery engine ensures flawless timestamp generation, the timing gate must still manage structural communication failures, such as prolonged Wi-Fi unavailability or total server dropouts.

A completely lost message is identified when the background network task fails to receive an HTTP `200 OK` acknowledgment from the server. This recovery path requires a multi-layered timeout strategy:

- **Lower-Level TCP Retries:** The underlying network interface stack (lwIP) automatically handles packet delivery failures in the background. If an individual Wi-Fi frame collides or is dropped due to RF interference, the stack executes an automatic exponential backoff retry loop (retransmitting at roughly 200 ms, 1 s, 2 s, and 4 s).
- **Application-Level HTTP Timeouts:** To prevent a stalled server from permanently locking up the background worker task, an application-level HTTP timeout is enforced at 2500 ms. If the server fails to reply within this window, the gate closes the network socket, flags a transmission error, and retains the event to allow subsequent items in the FreeRTOS queue to process.

Passive Retry Pacing and the Hard Watchdog Circuit

Rather than saturating the RF space during an active network drop, the system divides connection recovery into two distinct states monitored inside the primary `loop()` thread:

- **Passive Retry Pacing (Standard Outage):** If the connection drops but the total outage window remains under 15 seconds, the gate executes quiet, passive retry print tracking paced at 3000 ms intervals.
- **Hard Stack Watchdog Reset (Zombie State):** If the Wi-Fi status remains disconnected for **more than 15 seconds**, the firmware assumes the underlying wireless stack has entered a fatal zombie state or suffered a lockup. The system fires a hard network teardown command:

```
WiFi.disconnect(true, true); // Turns off the radio and completely purges  
saved credentials
```

After a 500 ms physical hardware cooling delay, the radio interface is completely rebuilt from scratch and issued a fresh `WiFi.begin(ssid, password, 0, bssid)` command to re-establish communications.

Real-Time Visual Backlog Indicator

An external indicator LED (`LED_PIN = 2`) provides real-time diagnostic status on the track without requiring a serial connection. By tracking the state transitions of

`uxQueueMessagesWaiting(networkQueue)` via a static edge-detection variable, the pin changes state instantly:

- **LED remains OFF:** Normal operations. Data is offloading faster than it is arriving.
 - **LED stays solid ON:** The network link is down or the server is stalling. Data packets are building up in RAM, indicating that the network queue is stuck.
-

Gate Present Heartbeat & Diagnostics

It is possible for a gate to fail or drop out entirely during a contest due to a sudden power failure, localized RF interference, or because the radio temporarily lost contact with the Access Point (AP). To detect these edge faults proactively, each gate transmits an explicit keep-alive heartbeat message (`"HB"`).

To prevent synchronized network collisions where multiple nodes inadvertently flood the access point simultaneously, the heartbeat relies on a unique prime number interval. A FreeRTOS software timer handles the delivery loop, firing exactly every 5147 ms.

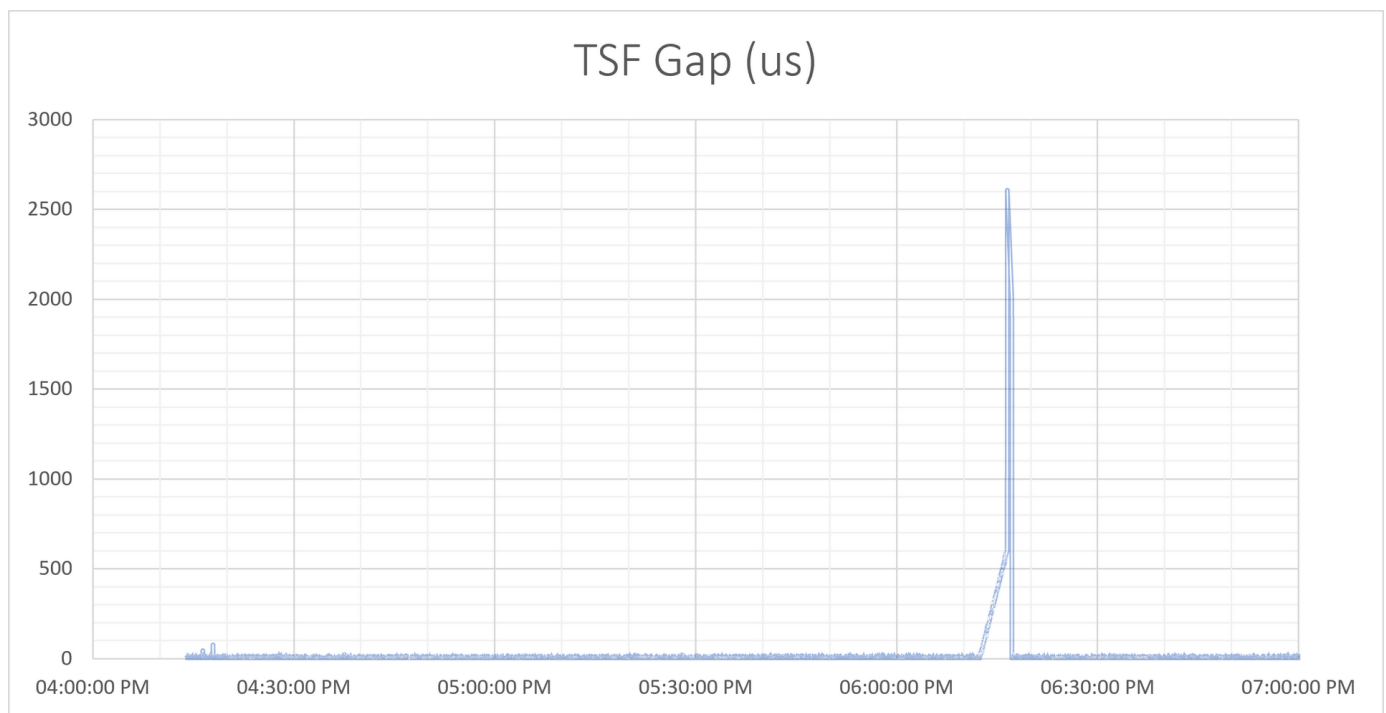
The master server ledger monitors the presence of these incoming heartbeats. If a gate falls silent for a prolonged window exceeding 15 seconds (matching the gate's internal hard reset threshold), the server immediately flags a system fault to notify the operator.

Field trials demonstrate that a gate may drop out for several seconds after losing contact with the server, but because events remain protected inside the asynchronous queue, it will seamlessly dump its backlog in an automated rapid catch-up burst the absolute millisecond the link re-establishes.

Test Results

One trial conducted over a period of approximately 3 hours collected 11,630 pairs of gate data at a continuous cadence of one pair per second. Throughout this evaluation window, there were zero mismatched or dropped messages. The average hardware gap between the times recorded by the gates was 18.4 μ s. While this indicates impressive system-wide consistency, the overall metric conceals an insightful observation.

Below is the plotted telemetry showing the hardware timing gap measured across the duration of the test:



As illustrated, at approximately 6:10 PM, the calculated gap began to increase perfectly linearly. Shortly thereafter, the gap stepped sharply to roughly $2000\ \mu\text{s}$ before steadily and linearly declining until it returned to near-zero.

The mechanism behind this excursion is that one of the remote gates missed consecutive beacon frames, causing its uncorrected local TSF clock to drift freely on its quartz crystal. Eventually, the variance relative to the expected baseline exceeded the threshold, prompting the timeline audit to reject the volatile network clock and engage the internal monotonic processor clock.

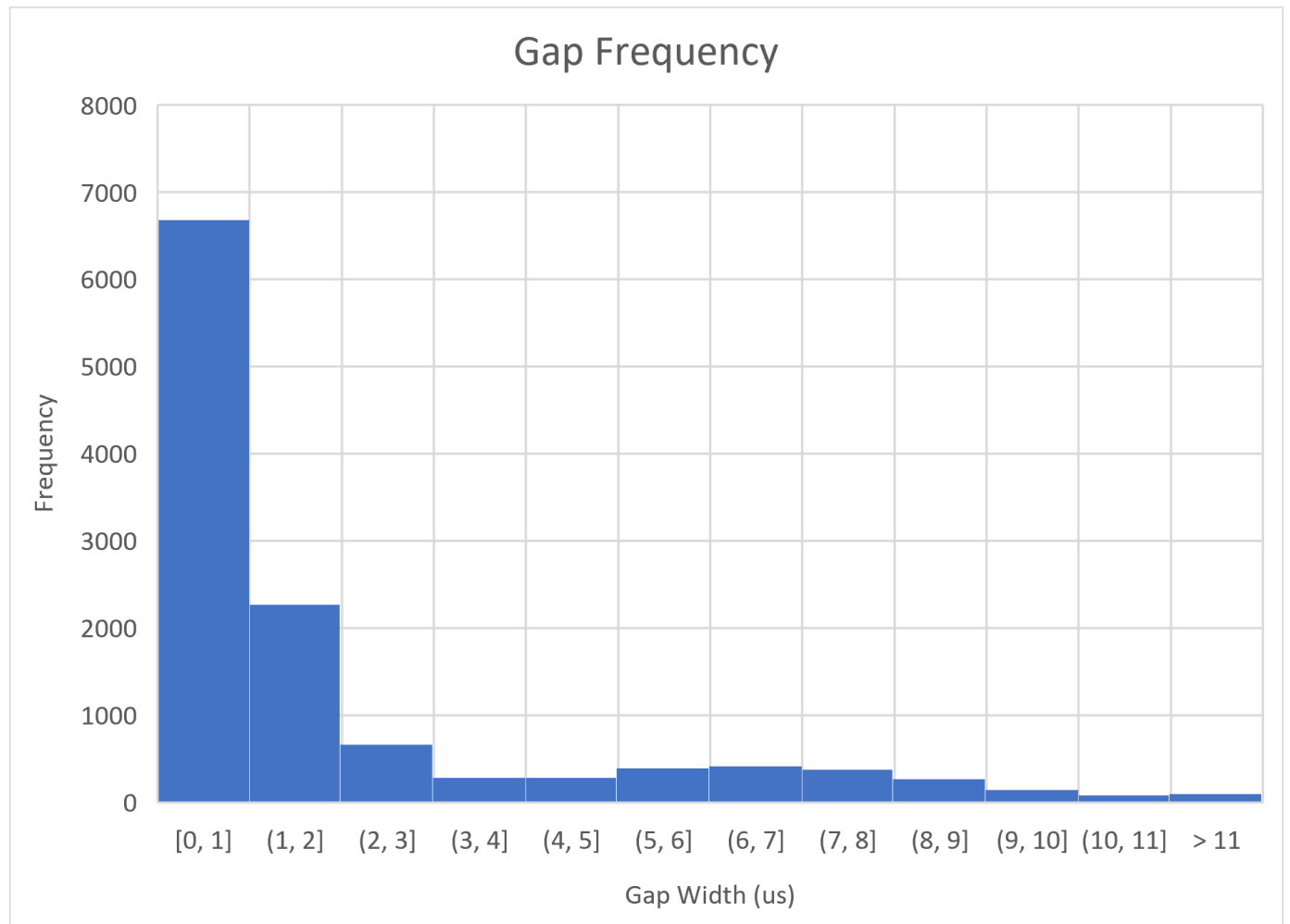
Because the processor clock's drift rate operated on a different slope relative to the alternate gate's clock, the error began to linearly converge. Once the affected gate intercepted a clear beacon frame from the Access Point, it successfully resynchronized its hardware TSF register, satisfied the sanity audit, and seamlessly returned control to the native TSF timeline - instantly restoring near-zero alignment.

The entire temporal excursion lasted just under 5 minutes, and the total drift was securely capped by the system. Consequently, even if a target had tripped the physical sensors during the absolute worst-case peak of this network blackout, the recorded runtime would have been protected and altered by no more than $2\ \text{ms}$.

Discounting this single network-induced excursion, the true average TSF tracking gap across the 3-hour trial was an astonishing $1.73\ \mu\text{s}$, and never exceeded $77\ \mu\text{s}$.

Following that test, the additional protections for tracking clock drift and unexpected outages were added and the gates were connected to a more stable dedicated access point. On a domestic mesh network, there is a tendency for the access points to reconfigure arbitrarily during a long test. However, when connected to a separate, stable portable router, the results are very consistent. Using the same 1PPS

simultaneous trigger on two timing gates, the gap between the recorded times from each gate is recorded over a three-hour period (approx 11,000 triggers) and summarized in a histogram:



Note that the times are in microseconds. During that period, the mean gap was $2.36 \mu s$ and the maximum gap was $55 \mu s$. There were no lost notifications, no need for the synthetic time and no cumulative drift. There was one occasion wher one of the gates was unable to send its notification. However, the event remained queued and was send a little later. The timing information remained intact.